

- **Name:** Muhammad Saad Azam
- **Roll Number:** 2025-MSCS-223

Papers

Paper 1: LoRA: Low-Rank Adaptation of Large Language Models

Summary

The paper introduces a new method to fine-tune Large Language Models (and other models in general, as the method itself is model agnostic). The model introduces injection of low rank decomposed matrices with trainable weights in a pretrained model. The pretrained weights are frozen. The benefit over other fine-tuning methods (like prefix tuning and adapter layers) is that:

- There is practically no inference delay.
- There is no compromise on sequence length.
- Saving minimal weights for different downstream tasks is still the better way to go.

$$h = W_o x + \Delta W x = W_o x + B A x$$

Objectives

The goals of the paper were to:

- Reduce cost of fine-tuning for Large Language Models.
- Introduce no to minimal inference delay
- Testing the hypothesis that major updates lie in the low-rank subspace.
- Compare LoRA with other fine-tuning mechanisms (adapter layers and prefix tuning)
- Avoid storing different weights for fully fine-tuned large model for different tasks. Instead, introduce lightweight, low rank, trained weight matrices and use those for different downstream tasks.

Key Findings

- Parameter fine-tuning works extremely well.
- Low-rank weights are expressive enough for numerous, domain-like tasks.
- No loss in inference efficiency if weights are merged.
- LoRA works over different models (like GPT-2, BERT etc.)
- Good multi-task potential

Limitations

- The paper highlights that some inference delay is introduced if there are multiple sets of weights for different downstream tasks. Merging is not easy if that is the case which eventually introduced some inference delay.
- Eventually, if the downstream task is entirely different in its domain relative to the original pretrained model learnt dataset, full fine-tuning will be required.

Paper 2: QLoRA: Efficient Finetuning of Quantized LLMs

Summary

Essentially, the paper establishes that using double quantization, paged optimizers and NF4 (NormalFloat4) data structure, big LLMs (like 65B parameters big) can be finetuned quite efficiently with minimal resources. With frozen quantized weights of the model, we then have the freedom to apply bigger rank LoRA matrices that are learnable, making them more flexible and extending their finetuning abilities.

Objectives

- Minimize memory footprint, decrease memory spikes for finetuning without sacrificing performance of models.
- Answer if LoRA benefits from quantized weights.
- Evaluate the existing benchmarks: Whether good performance in one benchmark translates to good performance in other?
- Pushing the capabilities of chatbots via finetuning using QLoRA and evaluate their performances.

Key Findings

- Double Quantization greatly reduces the memory footprint.
- NF4 is quite competent in performance to 4-bit finetuning.
- Quality of the dataset matters greatly for finetuning.
- Major memory footprint for training LoRA comes from the activation gradients. Using gradient checkpointing, we can greatly reduce this memory overhead as well.
- QLoRA finetuning can be competitive to full finetuning on quite a number of tasks.

Limitations

- Not established if QLoRA matches performance of full 16-bit finetuning of the 65B model.
- Limited evaluation on different benchmarks. The authors have left this evaluation to future work.

- Performance of the benchmarks depends how similar finetuning dataset is to the benchmark dataset.
- Broader evaluation of Guanaco model in chatbot environment is left for future work.
- Different bit presentations were not tried. For example, 3-bit GPTQ finetuning may match the same performance of QLoRA.

Paper 3: Enhancing Low-Rank Adaptation with Structured Nonlinear Transformations

Summary

LoRA lacks in expressiveness because of linearity. The paper introduces LoRAN which uses non-linear transformations to vanilla LoRA with the goal of improving expressiveness and complex feature learning from finetuning. The paper further introduces sinter, a scaled sine interference activation function which performed better than other activation functions on the tested benchmarks.

$$Sinter(x) = A \cdot \sin(\omega x) * x + x$$

Where A and ω are scalar constants and $*$ is element wise product.

Objectives

- Improve expressiveness of vanilla LoRA.
- Gain performance boost without introducing any new parameters, complexity factors.
- Analyze different activation functions and how they perform.
- Introduce sinter, a better activation function for LoRA.
- Attempt to explain why LoRAN has performed better than QLoRA and vanilla LoRA.

Key Findings

- LoRAN has consistently performed better than QLoRA and vanilla LoRA (performed even better on vanilla LoRA because of no quantization introduced by QLoRA)
- Sinter is robust enough and performed well against ReLU, Tanh and other activation functions.

Limitations

- Sinter does introduce some perturbed noise which can lead to training instability.
- While LoRAN does not introduce any new parameter, calculation of sinter does introduce slight computation overhead.

Paper 4: Transformer Layers as Painters

Summary

LLMs because of their immense size lack interpretability. This paper attempts to explain some relationship among the layers and how certain operations can affect the architecture of the model.

The analogy:

- Each layer is a painter that adds something to the the canvas (input)
- This means, some painters can work in parallel without sacrificing the performance. Similarly, some painters can also be reordered.
- Some painters may learn other painters skill and can be combined into one painter.
- However, the same painter cannot learn every single skill and eventually, the performance on canvas will degrade.
- If the task is step sensitive, like mathematical reasoning, shuffling painters will yield worse results.

Objectives

Answer the following questions:

1. Do layers use the same representation space?
2. Are all the layers necessary?
3. Are middle layers all doing the same function?
4. Does the layer order matter?
5. Can we run the layers in parallel?
6. Does order matter for some tasks more than others?
7. Does looping help parallelized layers?
8. Which variants harm performance the least?

Key Findings

- The middle layers do share same representation space to some extent.
- But they do not do the same task, as evident once the central layer was used for every single middle layer, which yielded degraded performance.
- Some layers can be skipped in the middle if they are sharing quite similar representation space.
- Reordering does not hurt catastrophically. However, for sequential, reasoning based tasks, order yields worse results on benchmarks. So, order does matter somewhat.
- Layers can be executed in parallel, with the assumption that they are doing separate tasks.

Limitations

- The paper lacks some practicality - the results could've explained which layers to target for finetuning.
- Why switching layers hurts sequential tasks is something underexplored.
- The findings may not apply the same for all families of models.

Paper 5: AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning

Summary

One limitation of LoRA is, the value of r is hardcoded. AdaLoRA attempts to solve this problem by designating r to each layer and then pruning it. This automatic pruning basically signifies importance of certain LoRA matrices, and which layers should be designated for LoRA. This is achieved by SVD-like decomposition of LoRA matrix in $P\Lambda Q$, where the singular values are then zeroed out. This is done carefully by smoothening gradients with uncertainty factor, with regularization to ensure orthogonality.

Objectives

- Unequal and dynamic reallocation of rank among layers.
- Parameter efficiency at lower budget.
- Dynamic rank adjustment during training.

Key Findings

- AdaLoRA consistently matched or even outperformed full finetuning and other PEFT methods.
- AdaLoRA was likely to designate more rank to FFN and top layers.

Limitations

- AdaLoRA was tested on very limited amount of models. It was not tested on variety of models.
- There is only rank pruning in AdaLoRA, no rank growth.

Paper 6: BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models

Summary

Finetuning only the bias terms of a model yields quite impressive results, training only about 0.08% of the parameters of the model while freezing the weights. This further pushes the

hypothesis that finetuning is more about exposing/resurfacing/unlocking the information that models possess instead of relearning everything.

Objectives

- Finetuning models within limited constraints without sacrificing major performance.
- Parameter efficiency at lower budget.

Key Findings

- Query and MLP layers biases finetuning yielded the best results when finetuned in conjunction.
- Comparable results to full finetuning of certain benchmarks.

Limitations

- The paper was tested on BERT only, not any other models.
- Task dependent narrative was not pushed sufficiently. Are the performance gains uniform across all the benchmarks? Why or why not?
- Many modern models do not have biases in their attention. This lack of bias terms yields BitFit somewhat limited in practicality.

Paper 7: LoRA vs Full Fine-tuning: An Illusion of Equivalence

Summary

Compared to full finetuning, LoRA introduces intruder dimensions that results in more loss (forgetting). Due to these new dimensions, the resultant solutions reached by full finetuning and LoRA are different. Despite having similar performance in certain benchmarks, the solution space they reach is quite different. This has implications in 'forgetfulness', relearning parameters and continual learning.

Objectives

- Draw comparison between LoRA, different variants of LoRA and how do they compete against full finetuning.
- Introduce the concept of intruder dimensions, and how these intruder dimensions differentiate the finetuning approaches.
- Attempt to explain these intruder dimensions and why are they introduced and what are their implications.

Key Findings

- LoRA forgets less, but that is not always the case.
- The intruder dimensions increase in magnitude as fine tuning progresses.
- Increase in intruder dimensions results in increase in forgetting.
- Full finetuned models have no intruder dimensions.
- Accumulating intruder dimensions hurt LoRA when it comes to continual learning.
- Freezing matrix A and appropriate choice of α can potentially decrease the number of intruder dimensions.

Limitations

- Reliance on SVD. Probably more interpretability metrics could've been used.
- Some more exploration of intruder dimensions when it comes task dependent tasks.
- The choice of threshold for calculating intruder dimensions is somewhat vague.

Summary List

- **LoRA: Low-Rank Adaptation of Large Language Models**
 - **Authors:** Edward Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, Weizhu Chen
 - **Link:** <https://arxiv.org/abs/2106.09685>
 - **Year:** 2021
 - **Problem Addressed:**
 - Finetuning is a resource intensive process. LoRA assumes that task updates are low-rank. So, we can decompose matrix and learn weights for that low-rank decomposed matrix.
 - Prefix tuning and adapter finetuning have their benefits but trade-offs too. Adapter finetuning introduces inference delay, prefix tuning sacrifices sequence length.
 - **Methodology:** Introduction of Low Rank decomposition matrices with trainable weights, while the pretrained model weights will be frozen.
 - **Dataset:** GLUE (General Language Understanding Evaluation) primarily used for benchmarking.
 - **Key Results:**
 - Substantially faster fine-tuning without sacrificing inference latency.
 - Multi-task trained matrices for different downstream tasks.
 - Low rank subspace is quite efficient for multitude of tasks within the domain, and achieve quite good results.
 - **Research Gaps:**
 - Which layers should be LoRA'ed? Where is performance the best?

- Some inference delay is introduced if multiple downstream tasks are fine-tuned for the model. Switching between the matrices is where the inference delay happens.
- If the learning domain entirely changes, fully fine-tuned training is eventually necessary - something highlighted in "LoRA Learns Less and Forgets Less" (Biderman et al., 2024)
- **QLoRA: Efficient Finetuning of Quantized LLMs**
 - **Authors:** Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, Luke Zettlemoyer
 - **Year:** 2023
 - **Link:** <https://arxiv.org/abs/2305.14314>
 - **Problem Addressed:**
 - The 16-bit precision consumes a lot of memory. Can quantization yield good performance by reducing the memory without sacrificing performance?
 - With quantization, LoRA can use bigger rank matrices for finetuning, further enhancing its capabilities.
 - How do the different benchmarks evaluate these models? Can we improve these benchmark tests for chatbots for more robust evaluation?
 - **Methodology:** Introducing NF4, Double Quantization and Page Optimization to greatly boost the finetuning, where base model weights are quantized and frozen while LoRA matrices are finetuned.
 - **Dataset:** OASST1, HH-RLHF, Super-Natural Instructions, Alpaca datasets and other benchmark datasets.
 - **Key Results:**
 - Faster finetuning with minimal sacrifice of performance.
 - NF4 with double quantization greatly reduces memory footprint.
 - LoRA matrices greater memory cost comes from activation gradients, not the weights themselves.
 - Chatbot evaluation is quite dataset dependent.
 - **Research Gaps:**
 - Not established if the performance matches full finetuning.
 - Limited evaluation on other chatbot benchmarks.
 - Not investigated other bit precision points. For example, instead of 4-bit, 3-bit GPTQ may also match the performance.
- **Enhancing Low-Rank Adaptation with Structured Nonlinear Transformations**
 - **Authors:** Guanzhi Deng, Mingyang Liu, Dapeng Wu, Yinqiao Li, Linqi Song
 - **Year:** 2025
 - **Link:** <https://arxiv.org/pdf/2509.21870>
 - **Problem Addressed:**

- LoRA works in limited subspace and is linear. Can we introduce more non-linearity for better performance?
- **Methodology:** Finetuning models Llama 7b, Falcon 7b, Flan-T5-Large, Llama-2 13b. SVD analysis, performance analysis and more.
- **Dataset:** Newgroups and SAMSuns
- **Key Results:**
 - Sinter introduces non-linearity that results in better performance than other activation functions.
 - LoRAN performed better than QLoRA and vanilla LoRA, without introducing any new variable.
- **Research Gaps:**
 - Sinter does introduce some computational expense for calculation.
 - Sinter also introduces some perturbed noise that can lead to some instability during training.
- **Transformer Layers as Painters**
 - **Authors:** Qi Sun, Marc Pickett, Aakash Kumar Nain, Llion Jones
 - **Year:** 2024
 - **Problem Addressed:**
 - LLMs lack interpretability because of their size. However, some layer-wise analysis and some operations can be done to explain how some layers can work, and what kind of operations can yield to what kind of performance.
 - **Methodology:** Frozen analysis of encoder-only (BERT) and decoder-only (Llama) models. Testing them on then various benchmarks.
 - **Dataset:** GLUE, HellaSwag, ARC, GSM8K, Winogrande, Lambada for benchmarking.
 - **Key Results:**
 - The middle layers do share same representation space to some extent.
 - But they do not do the same task, as evident once the central layer was used for every single middle layer, which yielded degraded performance.
 - Some layers can be skipped in the middle if they are sharing quite similar representation space.
 - Reordering does not hurt catastrophically. However, for sequential, reasoning based tasks, order yields worse results on benchmarks. So, order does matter somewhat.
 - Layers can be executed in parallel, with the assumption that they are doing separate tasks.
 - **Research Gaps:**
 - The paper lacks some practicality - the results could've explained which layers to target for finetuning.

- Why switching layers hurts sequential tasks is something underexplored.
- The findings may not apply the same for all families of models.
- **AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning**
 - **Authors:** Qingru Zhang, Minshuo Chen, Alexander Bukharin, Nikos Karampatziakis, Pengcheng He, Yu Cheng, Weizhu Chen, Tuo Zhao
 - **Link:** <https://arxiv.org/pdf/2303.10512>
 - **Year:** 2023
 - **Problem Addressed:** How can we allocate the parameter budget adaptively according to importance of modules to improve the performance of parameter-efficient fine-tuning?
 - **Methodology:** Instead of hardcoding r , we can prune it. Tested on DeBERTaV3-base.
 - **Dataset:** GLUE
 - **Key Results:**
 - AdaLoRA achieves consistently improved scores when it comes to GLUE tasks. Even performing better than full finetuning in certain benchmarks.
 - AdaLoRA was designating more budget to FFN and top layers.
 - Hence, ranks should not be allocated uniformly.
 - **Research Gaps:**
 - Was only tested on DeBERTa-v3, not decoder-only LLMs.
 - Does not necessarily indicate when we should increase r though, only prunes.
- **BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models**
 - **Authors:** BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models
 - **Year:** 2021
 - **Link:** <https://arxiv.org/abs/2106.10199>
 - **Problem Addressed:** Similar to problems addressed in the other papers: Parameter efficient finetuning with minimal resources. The way discussed in this paper is to finetune the bias terms only.
 - **Methodology:** Training only second MLP and query layers of BERT Large.
 - **Dataset:** GLUE
 - **Key Results:**
 - Worse performance if only query or MLP layers were fine tuned independently. However, in conjunction, the results improved substantially.
 - Comparable performance to finetuning.
 - Pushes the hypothesis that finetuning is not about relearning, but resurfacing (or unlocking) the pre-existing knowledge that models possess.
 - **Research Gaps**

- Larger models were not tried.
- Encoder only model (BERT) was tested.
- Many modern LLMs don't utilize biases in their attention blocks, so BitFit won't work on those as expected and successfully.
- **LoRA vs Full Fine-tuning: An Illusion of Equivalence**
 - **Authors:** Reece Shuttleworth, Jacob Andreas, Antonio Torralba, Pratyusha Sharma
 - **Year:** 2024
 - **Link:** <https://arxiv.org/pdf/2410.21228>
 - **Problem Addressed:** Inherently, LoRA and full finetuning may reach the same benchmark score, but they do not learn the same solutions. How and why is explored in the paper.
 - **Methodology:** RoBERTa was fully finetuned and LoRAed with different values of r . Llama-7b was checkpointed from previous and analyzed.
 - **Dataset:** MNLI, QQP, SST-2, SIQA and more.
 - **Key Results:**
 - LoRA forgets less, but that that is not always the case.
 - The intruder dimensions increase in magnitude as fine tuning progresses.
 - Increase in intruder dimensions results in increase in forgetting.
 - Full finetuned models have no intruder dimensions.
 - Accumulating intruder dimensions hurt LoRA when it comes to continual learning.
 - Freezing matrix A and appropriate choice of α can potentially decrease the number of intruder dimensions.
 - **Research Gaps:**
 - Reliance on SVD. Probably more interpretability metrics could've been used.
 - Some more exploration of intruder dimensions when it comes task dependent tasks.
 - The choice of threshold for calculating intruder dimensions is somewhat vague.